Understanding Neural Networks from the Ground Up: A NumPy-Only Implementation Compared with a Framework Baseline - Update 2

Priyo Raihan	Sheanthea Akter	Marzia Anjum	Mustofa Nuruzzaman	Sayema Tasnim Aroni
ID: 2311692042	ID: 2413587642	ID: 2412117042	ID: 2231929642	ID: 2413241642

CSE 299, Section 3 - Group 4

Abstract— Neural Networks are one of the most plausible ways to build a machine learning model. It is a mathematical model which uses different formulas and techniques to produce a error correcting mechanism. That's how the machine learns and develops it's answers.

I. INTRODUCTION

We will build a neural network with three inputs, four neurons in the hidden layer and one output. We are gradually developing our model, continuing this process would yeild us a complete neural network in future. We are using Wisconsin Breast Cancer dataset [1], we will use limited datas from it and make a prediction producing forward pass mechanism and a error correcting back propagation [3]. We will run it over over multiple sets of data to check it's performance.

II. THEORY

The Wisconsin Breast Cancer Diagnostic dataset contains measurements used to distinguish malignant and benign cases [1]. In this implementation, three features are selected: *perimeter3*, *area3*, and *concavity3*. Each feature is standardized using its mean and standard deviation, and the target is encoded as M = 1 and B = 0.

For input vector x, the input-to-hidden weight matrix W01 has dimensions 3 x 4. The weighted sum is passed through ReLU, f(a) = max(0, a), to produce four hidden activations [2]:

$$h = ReLU(x\ W01)$$

The hidden-to-output matrix W12 has dimensions 4 x 1. Multiplying the hidden activations by this matrix gives one linear prediction:

$$y_hat = h\ W12$$

The prediction is compared with the true label y using squared error. The code accumulates this error over the dataset and divides by the number of samples to report mean squared error (MSE). Squared-error objectives and gradient-based learning are standard neural-network methods [2, 3]:

$$MSE = (1/N)\ sum\ (y_hat_i - y_i)\text{^}2$$

During backpropagation, the output error d2 = y_hat - y is propagated through W12 and multiplied by the ReLU derivative to calculate the hidden error. The weights are updated using gradient-based rules and learning rate alpha = 0.01 [3]. Training runs for 100 epochs and updates weights after each sample. The code has no bias terms and uses a linear output layer.

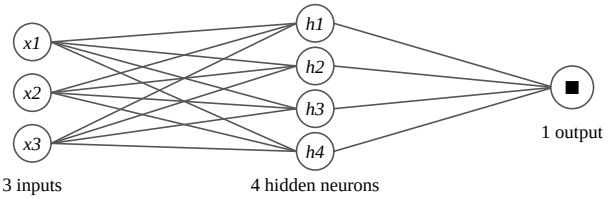

Fig. 1. Implemented architecture: three inputs, four hidden neurons, and one output.

III. IMPLEMENTED FEATURES

1) We loaded the dataset to obtain inputs and standardized them into same scale.

2) We built many foundational functions like prediction and hidden layer output returning function, activation functions, weightem sum determining function.

3) A proper forward pass mechanism which calculates weighted sum and applied activation functions on them which produces the hidden layers and the output.

4) A proper mean squared error calculating loss function.

5) A proper backpropagation which calculates dL/dw and and does gradient descent to adjust the weights [3].

IV. LIMITATIONS

1) We didn't used any biases in between input to hidden layer and between hidden layer to output, which is an important technical aspect of neural networks.

2) It still has only few neurons in the hidden layers and only limited inputs are used.

3) This model uses a simple loss function, more sophisticated ones aren't used.

The code also does not split the dataset into training and test sets or report metrics such as accuracy, precision, recall, or F1-score. Its linear output is not constrained to values between 0 and 1, which limits its interpretation as a probability.

V. UPCOMING IMPROVEMENTS

We will gradually increase the input features applied in our code. We will also increase the number of neurons in the hidden layer. So a more large number of datas would be calculated with proper mechanism thus making the system more sophisticated. We will use more sophisticated loss functions and would try to improve the backpropagation mechanism.

Future work can add bias terms, use a sigmoid output with binary cross-entropy, evaluate the model on held-out data, and report classification metrics [2, 4].

VI. REFERENCES

[1] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Machine learning techniques to diagnose breast cancer from fine-needle aspirates," Cancer Letters, vol. 77, no. 2-3, pp. 163-171, 1994.

[2] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533-536, 1986.

[4] UCI Machine Learning Repository, "Breast Cancer Wisconsin (Diagnostic)," University of California, Irvine. Available: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

The code's reported epoch error is the mean of per-sample squared errors. Because the output is linear, raw predictions can fall below 0 or above 1; a classification threshold is needed to assign a class. The current script also trains on all observations without a held-out test set, so performance on unseen cases remains unknown.